

Visual Stabilisation System with Real-Time Feedback on a Microcontroller Platform

Yuanyuan Tan

Abstract—This project presents a 2D visual orientation stabilizer implemented on a PIC18F87K22 microcontroller. The system estimates angular displacement using gyroscope BMI160 data and maintains a fixed inertial orientation on a graphical LCD display through a discrete feedback loop. Vertical movement is detected via a BMP388 pressure sensor, with partial support for radius correction.

The system was designed with three performance objectives: angular drift under 20° , vertical radius error within 1–3 cm, and a closed-loop update rate of 20–25 Hz. Experimental testing showed angular lag within 20° during 120° planar rotation and an update rate of 40.40 ± 0.01 Hz, confirming two design targets. While vertical correction logic was implemented, full testing was deferred. The results demonstrate a working closed-loop system with potential for more accurate integration, asymmetric correction, and full dimension expansion.

I. INTRODUCTION & MOTIVATION

This project explores a low-dimensional version of a visual stabilization system, where the displayed image remains steady even as the device moves. A familiar example is an electronic compass: the pointer stays fixed while the user rotates the device. Similar mechanisms underpin VR/AR systems, anchoring virtual elements to the real world.

The motion implemented by the system is two motions that can be implemented reliably on the PIC platform: xy-planar rotation and z-axis displacement. An orientation sensor gives information about angular velocity to estimate the heading. A pressure sensor gives an estimation of height. A manually triggered reset input sets the angle and height reference. Then, the angle and radius deviations are processed by a discrete feedback correction loop that adjusts the angle and radius of a simple GLCD plot.

The design is deliberately simple yet complete, forming a closed loop that supports future extensions. The three design objectives are: (1) angular drift under 20° , (2) vertical radius error under 1–3 cm, and (3) a closed-loop update rate of 20–25 Hz.^[1] While angular correction was fully implemented and tested, vertical stabilisation was treated as a secondary objective: core logic was implemented, but environmental compensation and live testing were reserved for future work (see Section III-C).

II. HIGH-LEVEL DESIGNS

A. Hardware Block

The core system is built around a PIC18F87K22 microcontroller, which interfaces with an orientation sensor (BMI160), a pressure sensor (BMP388), and a 128×64 -pixel GLCD display. Details are presented in product specifications. These components are selected to support the goal of stabilising an angle represented by points and a circle on the screen under user-induced rotation and vertical motion. This is shown in Figure 1, product rendered image.

The orientation sensor provides real-time angular velocity readings, while the pressure sensor estimates vertical movements. Together, they enable basic motion tracking. The reset button on the PIC sets the current angle as the new reference.

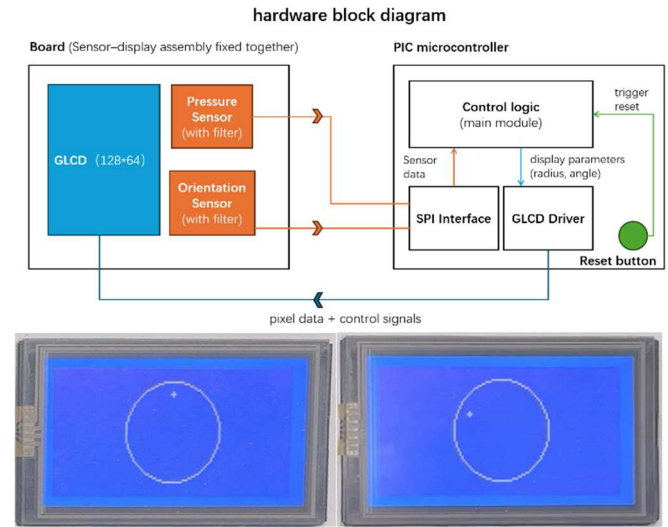


Fig. 1 hardware block diagram & product rendered image

The BMI160 and BMP388 are both connected via the SPI bus, with distinct chip select lines to ensure independent addressing. Because SPI offers higher data rates and fewer protocol constraints than I²C, it is suitable for real-time updates. The GLCD allows fast, direct pixel updates driven by the PIC. Overall, the hardware design provides just enough sensing and display capability to support closed-loop visual feedback, without unnecessary complexity.

The pressure sensor (BMP388) was originally included to track vertical displacement and adjust the circle radius dynamically. Although the sensor was wired and readable over SPI, full integration was later deprioritised due to the complexity of environmental temperature compensation and the relatively minor impact of vertical motion on perceived stability. The final system therefore fixes the radius at a calibrated value while retaining angular correction via the IMU.

B. Software Block

The main software design is shown in Fig.2.

The software begins with initialisation, moves through a fixed control loop, and end each cycle with GLCD display and UART data updates. Its primary goal is to convert raw angular velocity readings into clear, stable visual feedback on the GLCD.

After initial setup and reference capture, the system enters a continuous loop that drives the core functions. The gyroscope (orientation sensor) provides angular velocity data, which is

integrated over time (once per loop) to estimate the current orientation of the board.

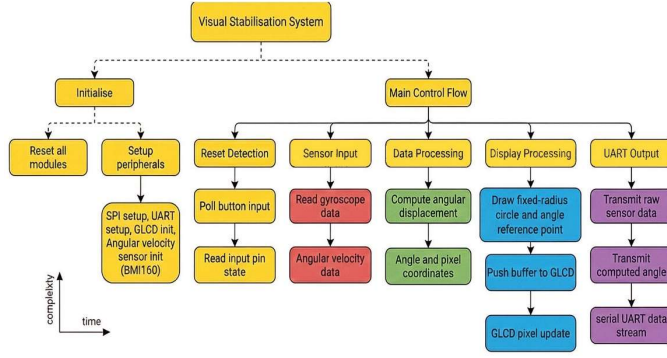


Fig. 2 top-down modular programming diagram

The software is organised into five modules: sensor interface (red), main control logic (yellow), data processing (green), UART interface (purple) and GLCD drawing (blue).

Rather than displaying this orientation as a full line, the system plots a single moving dot along a fixed-radius circle. This simple pattern make drawing easier and reduces the workload per frame. It also works better with the GLCD's slower optical response. A pixel buffer is used to stage each frame before display, this allows clean redraws without flickering. Incremental updates are chosen to preserve visual stability across frames.

Sensor data and computed angles are transmitted via UART in parallel with the display process. This provides a channel for real-time testing and performance analysis. An extension was planned to use pressure readings to adjust the circle's radius on the display.

Overall, the software is structured for simplicity and clarity. Each module complete its task independently, and the flow of execution remains the same across all frames.

III. SOFTWARE AND HARDWARE DESIGNS

A. Hardware Low-level Designs

Low-level hardware design is presented in Figure 3.

The PIC microcontroller operates from a 3.3 V supply, which is shared with the BMI160 and BMP388 sensors. The GLCD module is powered from a 5 V supply, as required by its internal driver circuitry and backlight. However, the GLCD's logic inputs are tolerant of 3.3 V signal levels, allowing direct interfacing without level shifting. With the configuration bits set to FOSC = HS1 and PLLCFG = ON, the system uses an external crystal oscillator with a PLL multiplier, resulting in an effective system clock in the range of 16–64 MHz.^[6]

Signal integrity was ensured by keeping wiring short and fixing all headers with adhesive to prevent loose connections. Both sensors are connected to SPI1, with shared SDI, SDO, and SCK lines, and separate software-controlled chip-select lines. BMI160 and BMP388 use standard 4-wire SPI.

The GLCD is controlled via an 8-bit parallel data interface, with additional signals including Enable (E), Data/Instruction (DI), and Read/Write (RW) lines. All control signals are managed directly through software. The GLCD pixels have a non-square aspect ratio (0.42 mm wide $\times$ 0.58 mm tall)^[4], which causes circular shapes to appear vertically stretched on screen. This distortion is purely visual and does not affect the logical accuracy of angle calculations or geometric operations.

The reset input is connected to RJ1 and configured as active-high, and it is polled during each loop iteration. For

debugging, UART is used to transmit raw and processed data. The TX pin is configured at 9600 bps and connects to terminal. No encoding errors or baud mismatch were observed during prolonged testing.

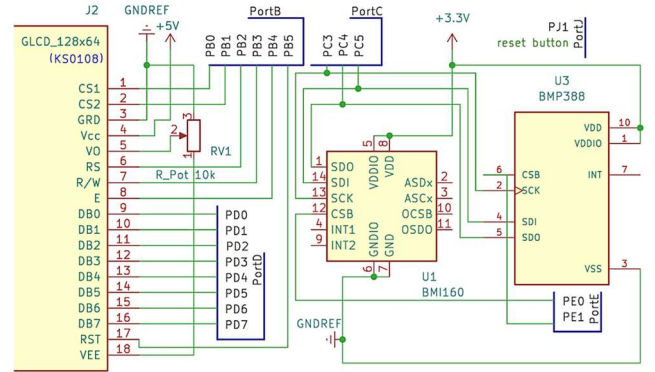


Fig. 3 Low-level schematic of PIC interface with GLCD and sensors This schematic shows the connection between PIC I/O ports and three peripheral modules: a 128 $\times$ 64 GLCD display (adjusted via a 10 k Ω potentiometer), BMP388 pressure sensor, and BMI160 IMU via GPIO and SPI. Power rails are split between +5 V (GLCD) and +3.3 V (sensors and PIC). All grounds are shared.

B. Software Design Details

In software, each module was written in PIC assembly, tailored for the system's hardware constraints.

1) Main Flow Chat and Calculations

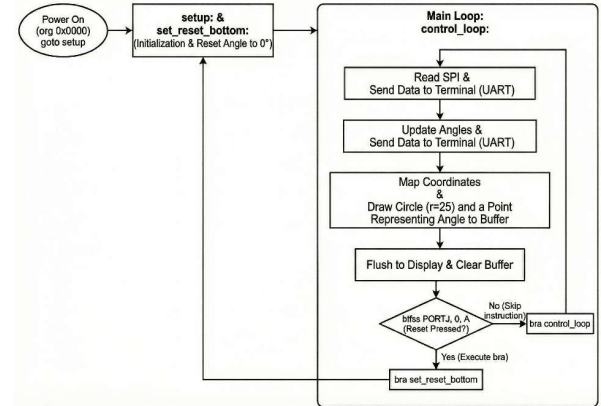


Fig. 4 main software flowchart

The main flowchart is shown in Figure 4. After initialisation, the program enters a fixed main loop. In each iteration, angular velocity data is read from the gyroscope via SPI and integrated to update the current angle. The angle is then mapped to screen coordinates, and a single point representing the direction is drawn along a fixed-radius circle in the display buffer. The buffer is flushed to the GLCD and cleared for the next frame. When pressed, the program resets the current angle to zero and restarts integration from the new position. Sensor data and computed angles are transmitted via UART for debugging purposes. A pressure-based radius update was originally planned but was removed due to timing constraints (see part C).

Raw 16-bit gyroscope outputs are linearly scaled to angular velocity and numerically integrated in software to estimate the rotation angle. In the current setup, raw gyroscope readings are scaled by a factor of 2^{12} using a 12-bit right shift. This gives an angular increment of about 0.02–0.05° per frame, which corresponds to 0.8–2° per second. This scaling factor was chosen based on tests to match the update interval (~24.7 ms) and typical angular speeds (50–200 °/s)^[3]. It gives a practical balance between sensitivity and display stability during moderate hand motion.

The accumulated angle is stored across three registers (16-bit integer and 8-bit fractional). Only the integer part is used for lookup table indexing, while the fractional part is kept to preserve precision below one degree during integration. A normalization step ensures the angle remains within $[0^\circ, 360^\circ)$, and sign correction is applied to maintain continuity across positive and negative rotations. This method avoids floating-point operations, and supports responsive visual feedback with minimal computational overhead.

With an 8-bit fractional representation, the smallest non-zero increment is $1/2^8 \approx 0.004$, corresponding to a precision on the order of 10^{-3} . This may cause a loss of small incremental updates during very slow rotational motion. Over extended periods of operation, these truncated increments can accumulate, leading to a gradual drift in the estimated angle.

2) SPI Interface and Sensor Settings

For the SPI interface, the system employs the PIC18F's built-in SPI module configured in Mode 00 (CKE=0, CKP=0, SMP=1) with 4-wire communication. This configuration ensures that the clock idles low, data is shifted on the falling edge, and sampled on the rising edge, and is fully compatible with both the BMI160 gyroscope and BMP388 pressure sensor. This is shown in Figure 5. The SPI clock was set to $F_{osc}/64$, offering a reliable data rate with minimal signal distortion. Moreover, the two sensors required distinct SPI routines. The BMI160 responds immediately after the command byte, while the BMP388 requires a one-byte delay (dummy byte) between the address and valid data.

The BMI160 gyroscope was configured for a 100 Hz output rate in normal mode, with an active digital filter pipeline and an oversampling factor of 2. This yielded an effective cutoff frequency of ~ 39.9 Hz, reducing high-frequency noise while preserving motion dynamics. The range was set to $\pm 250^\circ/\text{s}$, with a resolution of 131.2 LSB/ $(^\circ/\text{s})$.

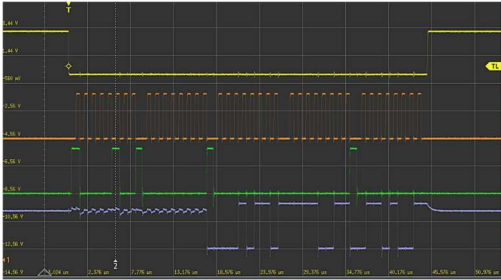


Fig. 5 SPI read data (BMI160)

A 4-wire SPI interface was adopted, where CS(yellow), clock(orange), PIC output-SDO(green), PIC input-SDI(purple).

3) Graphics Model

The GLCD graphics model is well designed. The GLCD used in this project has a resolution of 128 columns $\times$ 64 rows (8 pages $\times$ 8 pixels). To efficiently update the display, a 128 $\times$ 8-page buffer mechanism was implemented, where each element represents one display page. After converting the logical drawing coordinates (centred at (0, 0), with ranges $x \in [-64, 63]$, $y \in [-32, 31]$) to display coordinates ($x \in [0, 127]$, $y \in [0, 63]$), this separation of geometric logic from display-specific memory addressing improves code clarity and maintainability. After that, pixel updates are combined into the buffer using bitwise OR operations.

Once all drawing updates for a frame are complete, the entire buffer is written to the corresponding GLCD memory

pages in a single refresh, after which the buffer is cleared in preparation for the next frame. This approach significantly reduces the number of write operations to the GLCD and avoids repeated partial updates. As a result, visible flickering is minimised.

Circles are rendered using the Bresenham circle algorithm, where only one eighth of the circle is explicitly computed and the remaining points are obtained through symmetry in the logical coordinate space. This algorithm relies exclusively on integer arithmetic, avoiding floating-point operations, which makes it particularly well suited for microcontroller-based systems with limited computational resources, and easy to implement in assembly language.

For direction, a single dot moving along a circular path ($r=19$ pixels, circumference $= 2\pi \times 19 \approx 119.38$) is used. In this case, the circle is discretised into 3° steps ($360^\circ / 120$ per step), and the corresponding quantisation uncertainty is half a step, $\pm 1.5^\circ$, which is suitable for an aimed 10 deg deviation. The dot positions are stored in a lookup table to provide an easy and fast way to obtain the angle rather than calculate cos/sin.

4) PIC Memory Allocation

Due to the limited on-chip memory of the PIC microcontroller (4kb of file register memory), careful memory allocation was required. Variables and temporary values were placed in the bank1 and bank12 regions to allow fast and convenient access without frequent bank switching. The page buffer (1 kb) was also stored in file register memory across four consecutive banks, ensuring efficient read-modify-write operations during frequent display updates. In contrast, lookup tables mapping angle values to pixel coordinates were stored in program memory, since these tables are read-only during looping and require a large contiguous memory region.

C. Z-axis Response Design and Limitations Designs

The system was originally designed to include vertical motion sensing using a BMP388 pressure sensor. The goal was to adjust the radius of the displayed circle to reflect movement along the z-axis. The sensor was fully wired and communicated reliably over SPI, and software routines were implemented for device initialization, raw pressure acquisition, and variable-radius circle rendering on the GLCD.

The BMP388 operated in Ultra High-Resolution mode with pressure oversampling $\times 16$, temperature oversampling $\times 2$, and an IIR filter coefficient of 4, limiting RMS noise to approximately 5 cm.^[2] These settings were theoretically sufficient for height-based control within a 20 cm range. However, we did not fully integrate it at this stage due to the complexity of temperature compensation.

Experimental observations shows that pressure readings were highly sensitive to temperature variations and airflow. In practice, fluctuations of approximately 0.75 Pa ($\pm 6.2\text{cm}$) were observed without explicit temperature compensation, even under stationary conditions. When combined with the intrinsic sensor uncertainty about $\pm 5\text{cm}$, the total uncertainty becomes too large relative to the intended operational range. This causes unstable visual behavior dominated by drift and jitter rather than true vertical motion. Although official temperature compensation code is available, its computational complexity in PIC assembly would significantly increase loop period.

Consequently, the radius was fixed at a default value, and the control loop focused exclusively on angular response. It significantly improving system stability and preserving core functionality. This design choice still allows for future extension.

IV. RESULTS AND PERFORMANCE

A. System Timing

The average main loop cycle time is measured to be 24.744 ± 0.004 ms. It is determined by using a debug pin triggered once per loop and measuring the period with an oscilloscope. This confirms a stable closed-loop update rate of 40.40 ± 0.01 Hz, comfortably above the minimum requirement for real-time responsiveness (20 Hz) and well within the SPI communication limits of both sensors.

The GLCD display module was measured to refresh at approximately 5 ± 0.5 Hz, it is sufficient for producing a visually continuous and stable image without flickering within a specific range of angular speed. Each frame corresponds to a 0.2 s interval. To maintain visually continuous motion, the angular change per frame should not exceed $\sim 30^\circ$, implying a safe operational range without flickering of approximately ± 150 deg/s.

B. Effective Operating Range of Angular Speed

To find the range of rotation speeds where the system works reliably, we measured the angle lag under different manual rotation speeds. In each trial, we rotated the board 90° clockwise, the total rotation duration was recorded, and the manual average angular speed was defined as the ideal rotation angle (90°) divided by the measured duration.

Although the hand rotation was not perfectly smooth or constant, using average speed is more useful than tracking every fluctuation. This is because what matters here is the overall mismatch between expected and displayed angles after the full rotation, not the instantaneous speed changes during motion.

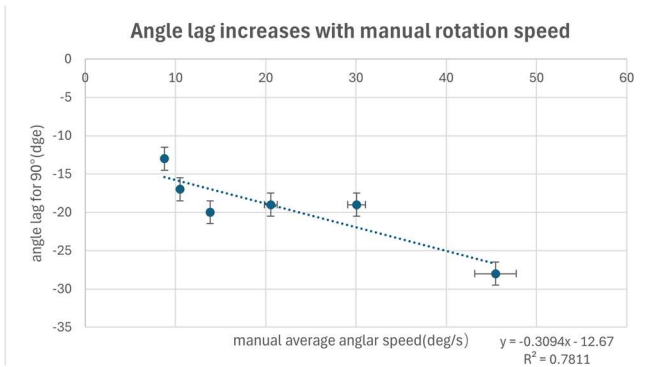


Fig. 5 the measured angle lag from intended 90° clockwise rotation against the average rotation speed.

Figure 5 shows the measured angle lag against the average rotation speed. Vertical error bars ($\pm 1.5^\circ$) represent the uncertainty of the final angle, explained in the Graphics Model section of the software design.

The horizontal bars show angular speed uncertainty based on total duration measurements. We excluded short-term fluctuations to focus on how average speed affects lag. The average angular speed was calculated as the total rotation angle divided by the trial duration. A linear regression is performed on the data. The resulting R^2 value was 0.78,

showing a strong correlation between rotation speed and accumulated angular deviation.

The overall trend in Figure 5 shows that angle lag becomes more negative as rotation speed increasing. At low speeds (around $5\text{--}10^\circ/\text{s}$), the measured angle still lags behind the intended 90° , but remains within an acceptable error range. As the rotation speed increases beyond $20^\circ/\text{s}$, the angle lag grows more quickly.

From the measured results, the angular deviation remains within an acceptable visual tolerance of $\pm 20^\circ$ for manual rotation speeds in the range of $5\text{--}20^\circ/\text{s}$. This range is chosen to balance both ends of the system's limitations. Below $5^\circ/\text{s}$ (2^2 times smaller than $20^\circ/\text{s}$). Although the angle lag is small, the fixed resolution limit of 0.004° per loop becomes dominant in the angular changing at such low speeds. This highlights how extremely low speed can lead to integration stall due to truncated loss. At this level, the angle may not visibly update during a slow rotation, leading to a flat or delayed response. This is caused by accumulated rounding losses in the digital integration process when the angular velocity per update is too small.

At the other end, speeds beyond $20^\circ/\text{s}$ produce visibly larger mismatches between the real and displayed orientation. This is because the system updates at a fixed rate. As rotation becomes faster, the angular change occurring within each update interval increases. Therefore, each update skips more detail, and the error accumulates over time. Even if the relative timing error per cycle stays the same, higher speeds produce larger absolute angle errors. This explains the near-linear correlation between rotation speed and lag seen in the regression.

So, $5\text{--}20^\circ/\text{s}$ is where we get the best trade-off between smoothness and accuracy. Note that this range refers to the trial average angular speed, calculated as the net rotation angle divided by total duration. Local speed deviations may fall outside this range, especially under manual input. Therefore, this definition serves to evaluate the system's long-term tracking behaviour. This defines the recommended working range for angular speed and sets the baseline for further performance tests in the next section.

C. Real-Time Angle lag with Manual Rotation

To evaluate the system's performance under typical hand held usage, a manual 120° clockwise rotation was performed. Although the motion was not actively controlled for speed, the average angular speed was $18.33^\circ/\text{s}$, within the identified stable range of $5\text{--}20^\circ/\text{s}$. (the raw angular velocity exhibits extended periods above $20^\circ/\text{s}$, but the slower speeds at the start and end ensure that the overall average remains within the target range).

Figure 6 shows the system's estimated angle (red) compared to the ideal linear trajectory (blue). The green dashed line shows the real-time angle lag. The red curve shows a smooth decrease with no sudden jumps. This confirms stable and continuous integration of angular velocity into the displayed angle. As expected, lag gradually accumulates as the rotation continues, reaching about 20° at the end of the 120° motion. In this range, both angle lag and responsiveness are maintained at an acceptable level. Moreover, the lag remains under 10° during the first half of the movement (within 50°), which define a region of optimal tracking performance.

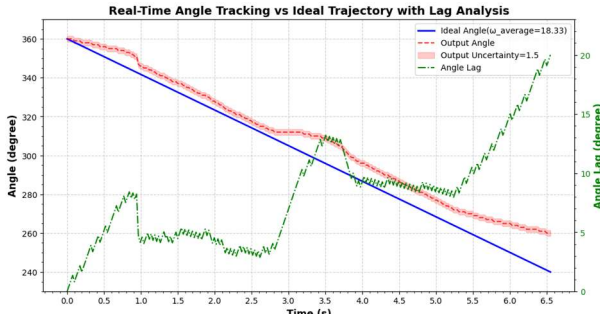


Fig. 6 Comparison between ideal angle trajectory (blue) and measured angle (red) during a manually controlled 120° clockwise rotation.

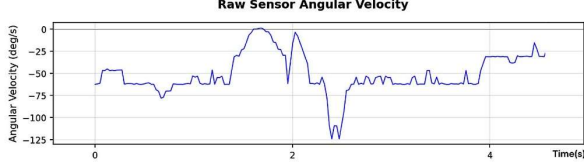


Fig. 7 Raw gyroscope output (z-axis angular velocity) recorded during the same manual rotation as in Figure 6, shown here from 1.5 s to 5.5 s.

The time intervals 0–1 s and 2.75–3.75 s in the red line corresponds to periods where output angle changes slow down or stall. These regions match the low-speed segments seen in the raw angular velocity plot (Figure 7). This suggests that temporary increases in lag are mainly due to manual input slowing down, not internal errors. Notice that the small speed fluctuations do not cause instability. The system recovers and resumes accurate tracking once the speed returns to normal.

Overall, the system is robust to non-uniform hand motion. As long as the average angular speed remains within the supported range, the angle output stays smooth, continuous, and within acceptable visual limits.

D. Operating Range

Based on the results presented above, the operating range of the system can be defined to clarify its performance boundaries. The angular velocity measurement range of the sensor is $\pm 250^\circ/\text{s}$. Exceeding this limit leads to saturation, causing trajectory discontinuities and loss of tracking stability. Within the sensor's valid measurement range, the system achieves the best balance between smoothness and accuracy when the trial average angular speed lies between 5 and $20^\circ/\text{s}$. This interval therefore gives the recommended operating range for long-term tracking behaviours. During practical operation, local speed fluctuations are unavoidable, and angle lag accumulates as motion continues. For a total rotation of 120° , the lag can reach approximately 20° . However, during the first half of the motion (within 50°), the lag remains below 10° . This corresponds to the region of optimal tracking performance.

We can test the system's tracking of vertical (z-axis) displacement using the same controlled method used for angular motion. These measurements define the practical operating boundaries of the system beyond stability and responsiveness alone.

V. UPDATES, MODIFICATION AND IMPROVEMENTS

A. Directional Asymmetry in Sensor Response

Angle lag increase with manual angular speed (CCW)

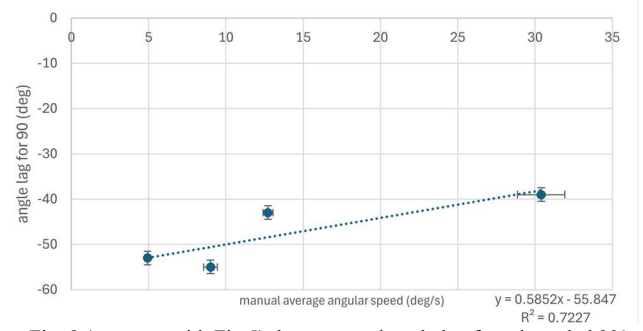


Fig. 8 (compare with Fig.5) the measured angle lag from intended 90° counterclockwise rotation against the average rotation speed.

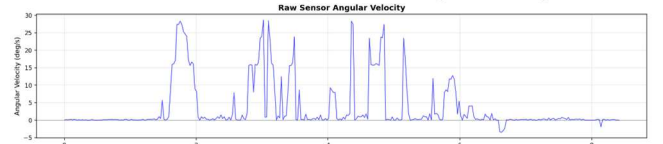


Fig. 9 (compare with Fig.7) Raw angular velocity comparison showing fragmentation in CCW trials. CW direction is negative velocity while CCW is positive.

Manual rotation tests reveal consistent asymmetry in system behavior depending on rotation direction. As shown in Figure 7, clockwise (CW) rotation produces continuous, well-defined angular velocity data and results in smooth, accurate angle estimation. In contrast, counterclockwise (CCW) trials often show fragmented angular velocity signals with frequent drops and spikes, shown in Figure 9. These cause visible lag and underestimation of the final angle. This discrepancy comes from systematic differences in sensor response, not from variations in user input. The issue stays the same no matter how we orient the gyroscope. Even changing its position doesn't help, so it's likely not due to mounting or alignment.

The fragmented CCW data causes integration error, as certain angular velocity samples are lost or distorted. So, cumulative angle estimation becomes less reliable. This is further illustrated in Figure 8, the CCW final angle lag plot. This trend is reversed unlike the clockwise case in Figure 5, larger average speeds lead to smaller final deviations. However, even at the lowest measured lag (-38°), the error remains well beyond the acceptable visual threshold of $\pm 20^\circ$. These directional differences persist even under similar average speeds and consistent manual conditions, suggesting a deeper hardware-level or communication-related origin.

To deal with this issue, we used different scaling factors for each direction when converting velocity to angle. CCW angular velocity was divided by 2^{14} , while CW used 2^{16} . Although the root cause of this asymmetry is not fully understood and does not appear to stem from the physical orientation of the gyroscope, this empirical adjustment significantly reduced directional bias, resulting in no visually noticeable asymmetry in the displayed motion. Future work could look into adaptive scaling or more detailed calibration to fix the root cause of the asymmetry.

B. Software Designs and Control logic improvements

The current angle estimation uses fixed-point arithmetic with a rough scaling factor, optimized for speed and efficiency. This, however, limits numerical precision. Dividing by binary values makes computation easier, but it limits both precision and flexibility. To improve this, future versions could use shift-add combinations to approximate

non-binary coefficients. They could also store more decimal places, for example by using 16-bit instead of 8-bit fractional representation. This would reduce rounding errors and better capture small or slow movements.

In addition, the integration method works like a proportional-only (P) controller, it reacts to angular velocity in real time but doesn't consider accumulated error or future changes. Adding elements like integral and derivative terms could help, such as incrementally increasing compensation for sustained lag, or slightly boosting angle updates when a sudden change in velocity is detected. These changes would improve both accuracy and visual smoothness.

VI. CONCLUSION

This project aimed to implement a low-dimensional visual stabilization system capable of maintaining angular drift under 20°, vertical radius error within 1–3 cm, and a closed-loop update rate of 20–25 Hz.

Based on integration test under manual planar rotation, the system successfully maintained angular lag within 20° during a 120° continuous rotation at an appropriate speed range (5–20 °/s). The measured update rate of 40.40 ± 0.01 Hz meets the design target. Vertical correction logic using barometric pressure was partially implemented in hardware and software, but full testing was deferred due to the complexity of environmental compensation and its limited impact on perceived visual stability.

The system also showed asymmetrical performance between clockwise and counterclockwise motion. This was mainly caused by non-uniform sensor output. We addressed it by empirically adjusting the velocity-to-angle scaling.

Overall, the project achieves a functional closed-loop stabilization pipeline, including sensing, integration, display control, and reset functionality. Future improvements may include higher precision numerical methods, asymmetry error correction, and stabilization with vertical feedback.

REFERENCES

[1] Y. Tan, *Two-Degree-of-Freedom Visual Stabilisation System on the PIC18*, Microprocessor Lab Project Proposal, unpublished internal document, Imperial College London, 2025.

[2] Bosch Sensortec, *BMP388 Digital, Barometric Pressure Sensor – Datasheet*, pp. 1–2 (absolute/relative accuracy, noise), 2017.

[3] TDK InvenSense / MikroElektronika, ICM-20600 6-axis IMU – Gyroscope and Accelerometer Specifications, accessed 2025.

[4] Winstar Display Co., *Graphic LCD Module WDG0151-TMI-V#N00 – Specification*, 2009.

[5] Microchip Technology Inc., PIC18F87K22 Family Datasheet: 64/80-Pin, High-Performance, 1-Mbit Enhanced Flash MCUs with 12-Bit A/D and XLP Technology, DS39960D, 2013. [Online]. Available: <https://www.microchip.com/en-us/product/PIC18F87K22>

PRODUCT SPECIFICATIONS

1. System Overview
The system is a high-precision microcontroller-based visual orientation indicator. It performs real-time Z-axis rotation estimation using gyroscope data integration, visualizing the motion as a single point on a circular path via a 128×64 GLCD.

“Microprocessor Project 2 – Device Code Repository,” GitHub, Available: <https://github.com/yt2523/MICROPROCESSORPROJECT2>

2. Technical Specifications

Category	Details
Microcontroller	Microchip PIC18F87K22
Display	WDG0151-TMI-V#N00 (128×64 Monochrome GLCD, KS0108 controller)
Sensors	6DOF IMU 2 Click (BMI160), Pressure 5 Click (BMP388)
Communication	SPI (Sensors), Parallel GPIO (GLCD), UART (Debug/Monitoring)
Control Logic	Timer-based loop scheduling and integration-based angle estimation

3. Functional Capabilities
Real-time Visualization: Smooth circular motion tracking on GLCD.
Reference Management: Manual Z-axis zero-point reset via push-button.
Data Telemetry: Real-time UART output for system monitoring.
Expandability: Built-in interface for pressure-based radius modulation (currently dormant).

4. Memory Utilization
Program Flash: 2,310 bytes (1.8% of 128KB)
(Includes 336 bytes of angle table)
Data RAM: 1,069 bytes (27.7% of 3.8KB)
(Includes 1,024-byte GLCD frame buffer)
EEPROM: 0 bytes used.
Configuration: 7 words (100% utilized).

5. Timing and Performance
Sensor Sampling Rate: 50 Hz (20 ms interval).
Main Control Loop: 24.744 ± 0.004 ms (~40.40 Hz).
GLCD Refresh Rate: ~5 ± 0.5 Hz.
Gyroscope Sensitivity: 131.2 LSB/(°/s) at ±250 °/s range.
Angular Resolution: (0.0039°) per integration step.

6. Operational Constraints
Optimal Tracking Average Speed: 5 – 20 °/s.
Optimal Range at Optimal Speed: ±120°
Maximum Lag: 20° cumulative lag during a 120° continuous rotation.
Long time is not accurate
Zero-Point Calibration: Due to inherent gyroscopic cumulative drift, periodic manual resets are recommended during long-term operation to maintain angular precision.

APPENDIX

Use of AI Tools:
ChatGPT was used for grammar correction and language polishing. Figures 2 and 4 were generated with Gemini based on original content and design.